1. Who is the original creator and designer of the Python programming language?
- **(A)** Dennis Ritchie
- **(B)** Guido van Rossum
- **(C)** Bjarne Stroustrup
- **(D)** James Gosling

2. Python was officially released to the public in which year?
- **(A)** 1985
- **(B)** 1991
- **(C)** 1995
- **(D)** 2000

3. Which programming paradigm does Python inherently support?
- **(A)** Object-Oriented Programming only
- **(B)** Functional Programming only
- **(C)** Imperative and Procedural only
- **(D)** Multi-paradigm (OOP, Imperative, Functional)

4. How is Python source code (.py) executed by the standard CPython runtime?
- **(A)** Directly compiled to native machine code binaries
- **(B)** Compiled to intermediate bytecode (.pyc) and interpreted by PVM
- **(C)** Translated to pure assembly language beforehand
- **(D)** Executed directly by hardware CPU microcode

5. Which environment variable in Windows is crucial for executing 'python' from any command prompt?
- **(A)** PYTHONPATH
- **(B)** PATH
- **(C)** SYS_PATH
- **(D)** WINDIR

6. Which tool is the standard package manager included with Python installations to install third-party packages?
- **(A)** pip
- **(B)** npm
- **(C)** gem
- **(D)** cargo

7. What is the primary role of the Python Virtual Machine (PVM)?
- **(A)** To edit Python script files with syntax highlighting
- **(B)** To interpret intermediate bytecode instructions into machine actions
- **(C)** To link C++ object libraries into machine code
- **(D)** To convert Python into JavaScript for web browsers

8. Which command is used to launch an interactive Python REPL shell on Windows?
- **(A)** run python
- **(B)** python
- **(C)** pyexec
- **(D)** compile

9. What is the output of running print('Hello, World!') in Python 3?
- **(A)** Hello, World!
- **(B)** ('Hello, World!')
- **(C)** None
- **(D)** SyntaxError

10. In Python 3, why does print 'Hello' (without parentheses) raise an error?
- **(A)** print is a deprecated operator
- **(B)** print is a built-in function requiring parentheses
- **(C)** Quotes are illegal without parentheses
- **(D)** Single quotes are reserved for integers

11. Which of the following is a valid identifier (variable name) in Python?
- **(A)** 2nd_counter
- **(B)** _total_sum
- **(C)** total$sum
- **(D)** class

12. Why can the word 'while' NOT be used as an ordinary variable name in Python?
- **(A)** It contains five lowercase letters
- **(B)** It is a reserved Python language keyword
- **(C)** It is an existing mathematical constant
- **(D)** It conflicts with Windows registry keys

13. Which built-in function returns the data type classification of any Python object?
- **(A)** typeof()
- **(B)** kind()
- **(C)** type()
- **(D)** datatype()

14. Which built-in function returns the unique integer memory address identifier of an object?
- **(A)** address()
- **(B)** pointer()
- **(C)** id()
- **(D)** mem()

15. What type does the expression x = 42 have in Python 3?
- **(A)** int
- **(B)** float
- **(C)** long
- **(D)** Integer

16. What type does the expression y = 3.14159 have in Python?
- **(A)** double
- **(B)** float
- **(C)** real
- **(D)** decimal

17. What is the data type of the value True in Python?
- **(A)** Boolean
- **(B)** bool
- **(C)** int1
- **(D)** bit

18. Which built-in constant represents the absence of a value or null state in Python?
- **(A)** null
- **(B)** nil
- **(C)** None
- **(D)** void

19. Which operator is used for calculating the remainder of an integer division in Python?
- **(A)** /
- **(B)** //
- **(C)** %
- **(D)** ^

20. What is the result of 7 // 2 in Python 3?
- **(A)** 3.5
- **(B)** 3
- **(C)** 4
- **(D)** 3.0

21. What is the result of 7 / 2 in Python 3?
- **(A)** 3
- **(B)** 3.5
- **(C)** 3.0
- **(D)** 4

22. Which operator performs exponentiation (power calculation) in Python?
- **(A)** ^
- **(B)** **
- **(C)** ^^
- **(D)** pow_op

23. What is the character used to denote a single-line comment in Python?
- **(A)** //
- **(B)** /*
- **(C)** #
- **(D)** --

24. Which of the following creates a multiline string or docstring in Python?
- **(A)** /* ... */
- **(B)** <!-- ... -->
- **(C)** """ ... """
- **(D)** ~~ ... ~~

25. What is the result of the string concatenation 'Data' + 'Science'?
- **(A)** Data Science
- **(B)** DataScience
- **(C)** Error: cannot add strings
- **(D)** ['Data', 'Science']

26. What is the result of the string repetition operation 'Hi' * 3?
- **(A)** HiHiHi
- **(B)** Hi 3
- **(C)** Error: unsupported operand type
- **(D)** ('Hi', 'Hi', 'Hi')

27. Which built-in function measures the number of characters in a string?
- **(A)** count()
- **(B)** size()
- **(C)** length()
- **(D)** len()

28. What is the index of the first character in any Python string s?
- **(A)** 1
- **(B)** 0
- **(C)** -1
- **(D)** None

29. What does negative indexing s[-1] access in a non-empty string s?

(A) The first character
(B) The last character
(C) The second character
(D) IndexOutOfBoundsError

30. What error is raised when attempting to access a variable name that has not been defined?

(A) TypeError
(B) ValueError
(C) NameError
(D) AttributeError

31. Which of the following is an immutable data type in Python?

(A) list
(B) dict
(C) set
(D) str

32. Which module provides the list of all reserved keywords in the current Python interpreter?

(A) sys
(B) keyword
(C) builtins
(D) os

33. Which operator is used to test whether two variables point to the exact same object in memory?

(A) ==
(B) is
(C) equals
(D) eq

34. Which operator tests if two variables have equivalent content/values?

(A) ==
(B) is
(C) equals
(D) :=

35. What is the value of 2 + 3 * 4 in Python based on operator precedence?

(A) 20
(B) 14
(C) 24
(D) 10

36. What is the result of bool(0) in Python?

(A) True
(B) False
(C) None
(D) 0

37. What is the result of bool('False') in Python?

(A) False
(B) True
(C) None
(D) ValueError

38. Which symbol represents the bitwise XOR operator in Python?

(A) ^
(B) ~
(C) |
(D) &

39. Which symbol represents the bitwise NOT operator in Python?

(A) !
(B) ~
(C) ^
(D) -

40. Which file extension is standard for compiled Python bytecode files?

(A) .py
(B) .pyc
(C) .obj
(D) .class

41. What is the result of 2 ** 3 ** 2 in Python, considering exponentiation associativity?

(A) 64
(B) 512
(C) 36
(D) 72

42. What is the evaluated result of -7 // 2 in Python?

(A) -3
(B) -4
(C) -3.5
(D) -4.0

43. What is the output of 7 % -3 in Python arithmetic?

(A) 1
(B) -2
(C) 2
(D) -1

44. What is the evaluated output of the expression 10 / 2 + 3 * 2?

(A) 11.0
(B) 16.0
(C) 11
(D) 16

45. What is the output of print(type(1 / 1)) in Python 3?

(A) <class 'int'>
(B) <class 'float'>
(C) <class 'number'>
(D) <class 'rational'>

46. Consider a = [1, 2]; b = [1, 2]. What do (a == b) and (a is b) evaluate to?

(A) True, True
(B) True, False
(C) False, True
(D) False, False

47. For small integers x = 256; y = 256, what does (x is y) evaluate to in standard CPython?

(A) True (due to small integer interning -5 to 256)
(B) False (different memory allocations)
(C) SyntaxError
(D) None

48. What does the expression 3 < 5 < 10 evaluate to in Python?

(A) True (chained comparison)
(B) False
(C) TypeError
(D) 10

49. What is the output of print('Python'[1:4])?

(A) Pyt
(B) yth
(C) ytho
(D) Pyth

50. What is the output of print('Engineering'[::2])?

(A) Egnein
(B) Egienig
(C) Engin
(D) nein

51. What is the output of print('Algorithm'[-3:])?

(A) thm
(B) ithm
(C) hm
(D) Algorithm

52. What is the output of print('Python'[::-1])?

(A) nohtyP
(B) Python
(C) P
(D) SyntaxError

53. What does the membership test 'on' in 'Python' return?

(A) True
(B) False
(C) 1
(D) None

54. What happens when you execute s = 'Hello'; s[0] = 'J'?

(A) s becomes 'Jello'
(B) TypeError: 'str' object does not support item assignment
(C) ValueError: immutable character
(D) IndexError: string assignment

55. What is the result of 0.1 + 0.2 == 0.3 in standard IEEE 754 floating-point Python?

(A) True
(B) False (due to floating-point precision representation)
(C) None
(D) ZeroDivisionError

56. Which function in the math module accurately tests floating-point equality with a tolerance?

(A) math.equal()
(B) math.isclose()
(C) math.approx()
(D) math.same()

57. What is the value of 5 & 3 in Python bitwise arithmetic?

(A) 1
(B) 7
(C) 5
(D) 3

58. What is the value of 5 | 3 in Python bitwise arithmetic?

(A) 1
(B) 7
(C) 8
(D) 15

59. What is the output of 1 << 4 in Python?

(A) 4
(B) 8
(C) 16
(D) 32

60. What is the output of 32 >> 2 in Python?

(A) 16
(B) 8
(C) 4
(D) 2

61. What does the walrus operator (:=) introduced in Python 3.8 do?

(A) Compares values strictly
(B) Assigns values to variables as part of an expression
(C) Creates an immutable pointer
(D) Performs ceiling division

62. What is the output of bool([]) and bool([0])?

(A) False, True
(B) False, False
(C) True, True
(D) True, False

63. What is the output of True + True + False in Python?

(A) 2
(B) True
(C) TypeError
(D) 1

64. In Python, what is the output of '2' + '3' * 2?

(A) '26'
(B) '10'
(C) '233'
(D) '2323'

65. What is the output of 'apple' < 'banana' in Python?

(A) True (lexicographical comparison based on ASCII/Unicode values)
(B) False
(C) TypeError
(D) None

66. What is the output of the expression 0 or 5 and 10?

(A) 0
(B) 5
(C) 10
(D) True

67. Why does the expression False and (1 / 0) NOT raise a ZeroDivisionError?

(A) Short-circuit evaluation stops at False without evaluating the second operand
(B) Division by zero returns 0 in boolean contexts
(C) ZeroDivisionError is suppressed in logical expressions
(D) PVM rearranges operands

68. What is the output of True or (1 / 0)?

(A) True (short-circuit evaluation skips second operand)
(B) ZeroDivisionError
(C) False
(D) None

69. What is the result of not (5 > 3 and 2 < 1)?

(A) False
(B) True
(C) None
(D) TypeError

70. What is the output of int(3.99) in Python?

(A) 4
(B) 3 (truncates towards zero)
(C) 3.0
(D) ValueError

71. What does float('inf') evaluate to in Python?

(A) A special floating-point representation of positive infinity
(B) ValueError: invalid literal
(C) 0.0
(D) OverflowError

72. What does the expression complex(2, 3) create in Python?

(A) A complex number (2 + 3j)
(B) A tuple (2, 3)
(C) A coordinate object (2, 3)
(D) TypeError

73. What is the imaginary unit symbol in Python complex number literals?

(A) i
(B) j
(C) k
(D) im

74. What is the output of z = 3 + 4j; print(z.real, z.imag)?

(A) 3 4
(B) 3.0 4.0
(C) (3, 4)
(D) 7.0

75. What does the expression 'python'.capitalize() produce?

(A) 'PYTHON'
(B) 'Python'
(C) 'python'
(D) 'PyThOn'

76. What is the CPython GIL (Global Interpreter Lock)?

(A) A lock ensuring file I/O consistency
(B) A mutex preventing multiple native threads from executing Python bytecode simultaneously
(C) A mechanism preventing garbage collection memory leaks
(D) A compiler flag optimizing nested loops

77. In CPython, what is the primary automatic memory management mechanism?

(A) Generational garbage collection alone
(B) Reference counting supplemented by cyclic garbage collector
(C) Mark-and-sweep collector alone
(D) Manual pointer deallocation (free/delete)

78. When does CPython immediately reclaim heap memory occupied by an object?

(A) When its reference count drops to zero
(B) Only when the interpreter terminates
(C) When the heap reaches 100% capacity
(D) Every 60 seconds via cron

79. Which module allows developers to inspect and manipulate Python's cyclic garbage collector?

(A) gc
(B) mem
(C) sys.alloc
(D) cleanup

80. What does the __pycache__ directory contain?

(A) Log files from interactive REPL sessions
(B) Compiled bytecode files (.pyc) tagged with Python version
(C) Temporary cache of variable names
(D) Downloaded pip source packages

81. What does the shebang line #!/usr/bin/env python3 at the start of a script accomplish on POSIX systems?

(A) Specifies the interpreter path for direct script execution
(B) Imports the env module automatically
(C) Sets compiler optimization level 3
(D) Locks the script to Python 3.0 exclusively

82. What is PEP 8 in the Python ecosystem?

(A) The formal Python language specification for bytecode compilers
(B) The official Style Guide for Python Code
(C) The specification for Python package metadata
(D) The security protocol for PyPI packages

83. According to PEP 8, what is the recommended indentation standard for Python code?

(A) 1 tab character
(B) 4 spaces per indentation level
(C) 2 spaces per indentation level
(D) 8 spaces per indentation level

84. What is the result of executing sys.getrefcount(x) on a newly assigned integer variable?

(A) Always exactly 1
(B) At least 2 (the variable reference plus the argument passed to getrefcount)
(C) 0
(D) Memory address of x

85. What occurs when Python evaluates a = 1000; b = 1000 in separate REPL lines in standard CPython?

(A) a is b evaluates to True because all integers are interned
(B) a is b evaluates to False because 1000 is outside the small integer cache (-5 to 256)
(C) a == b evaluates to False
(D) NameError is raised

86. What is string interning in Python?

(A) Encrypting strings in bytecode
(B) Reusing identical immutable string instances in a lookup table to save memory and speed comparison
(C) Converting ASCII strings to UTF-32 automatically
(D) Embedding C strings into Python modules

87. What is the output of print('foo' 'bar') with adjacent string literals separated only by whitespace?

(A) 'foo bar'
(B) 'foobar'
(C) SyntaxError
(D) ('foo', 'bar')

88. What is the result of the expression ~5 in Python two's complement integer arithmetic?

(A) -5
(B) -6
(C) 4
(D) -4

89. Why does ~x equal -(x + 1) in Python?

(A) Python integers use two's complement binary representation of infinite precision
(B) Python uses one's complement representation
(C) It is an arbitrary design decision from ABC language
(D) Because bitwise NOT flips only the sign bit

90. What is the result of (True,) * 3 in Python?

(A) (True, True, True)
(B) True
(C) TypeError
(D) [True, True, True]

91. What does the expression divmod(17, 5) return in Python?

(A) (3.4, 2)
(B) (3, 2) representing (quotient, remainder)
(C) (2, 3)
(D) 17.0

92. What is the output of print(format(1234.5678, '.2f'))?

(A) '1234.56'
(B) '1234.57'
(C) '1234.5'
(D) '1235'

93. What does an f-string f'{x=}' produce if x = 42 (introduced in Python 3.8)?

(A) 'x=42'
(B) '42'
(C) '{42}'
(D) SyntaxError

94. What does the built-in function callable(obj) return?

(A) True if the object appears callable (like a function or class), False otherwise
(B) The number of parameters required by obj
(C) The memory location of obj
(D) None

95. What does id(None) represent during a single Python interpreter run?

(A) A constant unique integer memory address for the singleton NoneType object
(B) A random integer that changes every time None is accessed
(C) 0
(D) -1

96. In Python 3, what is the maximum value an int object can represent before triggering an integer overflow error?

(A) 2^31 - 1
(B) 2^63 - 1
(C) 2^127 - 1
(D) Arbitrary precision; bounded only by available system RAM

97. What does sys.byteorder return on modern x86/x64 Windows systems?

(A) 'big'
(B) 'little'
(C) 'network'
(D) 'undefined'

98. What does the expression round(2.5) evaluate to in Python 3 (banker's rounding)?

(A) 3
(B) 2 (rounds to nearest even integer)
(C) 2.5
(D) 3.0

99. What does round(3.5) evaluate to under Python 3's round-half-to-even rule?

(A) 4
(B) 3
(C) 3.5
(D) 4.0

100. What is the result of hash((1, 2, [3, 4])) in Python?

(A) A valid 64-bit integer hash
(B) TypeError: unhashable type: 'list'
(C) None
(D) 0

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the internal execution workflow of Python programs. Compare CPython with compiled languages (C/C++) and explain the roles of Bytecode and the Python Virtual Machine (PVM).

Q2. Discuss Python's Object Model and Dynamic Typing. Explain the three fundamental attributes of every Python object (Identity, Type, Value) with code demonstrations.

Q3. Explain CPython's Small Integer Caching (-5 to 256) and String Interning mechanisms. Write code to verify memory identity (is) versus value equality (==).

Q4. Explain Mutable vs Immutable data types in Python. Detail how memory allocation behaves during in-place mutations vs rebinding operations with illustrative code.

Q5. Explain Python Variable Naming Conventions (PEP 8) and NameError exceptions. Discuss common scenarios that trigger NameError and how to resolve them.

Q6. Analyze Python's Arithmetic Operators (+, -, *, /, //, %, **). Explain Floor Division (//) and Modulus (%) with positive and negative operands.

Q7. Explain Operator Precedence and Associativity in Python. Provide a full hierarchy table from highest to lowest precedence with illustrative evaluation examples.

Q8. Explain Logical Operators (and, or, not) and Short-Circuit Evaluation in Python. How does short-circuit evaluation optimize execution and prevent runtime exceptions?

Q9. Detail the representation and mechanics of Python Bitwise Operators (&, |, ^, ~, <<, >>). Explain two's complement integer arithmetic and why ~x equals -(x + 1).

Q10. Explain String Indexing and Slicing in Python. Detail the syntax s[start:stop:step], positive vs negative indexing, boundary defaults, and slice reversal.

Q11. Explain String Immutability and String Operations (+, *, in, not in). Why are strings designed to be immutable in Python, and how does this affect memory efficiency and security?

Q12. Discuss the built-in type conversion and type casting functions in Python (int, float, str, bool, complex). Explain explicit conversion versus implicit type coercion.

Q13. Explain Python Comments, Docstrings, and the Composition of Statements. How are docstrings accessed at runtime via __doc__ and help()?

Q14. Explain Python 2 versus Python 3 architectural differences. Focus on print, Unicode string handling, integer division, and range vs xrange.

Q15. Explain Python Virtual Environments (venv) and Package Management (pip). Why are virtual environments essential in enterprise software engineering?

Q16. Explain the Python Standard Library and the 'Batteries Included' philosophy. Provide examples of 5 essential built-in modules (math, sys, os, random, time).

Q17. Explain Modern Python String Formatting Paradigms: % formatting, str.format(), and Formatted String Literals (F-Strings). Compare performance and readability.

Q18. Explain CPython's Reference Counting and Cyclic Garbage Collection mechanisms. How are reference cycles detected and resolved?

Q19. Explain Python's sys.argv and Command-Line Argument Processing. Write a standalone Python script demonstrating command-line parameter parsing.

Q20. Explain Floating-Point Representation (IEEE 754) in Python. Why does 0.1 + 0.2 != 0.3, and how should financial/scientific calculations handle precision?

Q21. Design an interactive Python console application that computes Compound Interest and displays an annual amortization schedule table.

Q22. Implement a Python program for Temperature Conversion supporting Celsius, Fahrenheit, and Kelvin scales with input validation and error handling.

Q23. Implement a comprehensive Python script demonstrating all Bitwise Operations to manipulate hardware register control flags (Set, Clear, Toggle, Check).

Q24. Implement a Python program to perform Cartesian Coordinate System operations: Distance calculation, midpoint, polar coordinate conversion, and slope.

Q25. Explain the Walrus Operator (:=) introduced in Python 3.8. Discuss its motivation, syntax, performance advantages, and write code demonstrating its practical use.

Q26. Explain String Encoding and Decoding (ASCII, UTF-8, Unicode) in Python. Detail how str.encode() and bytes.decode() bridge text and binary data.

Q27. Explain the keyword module and dynamic inspection of Python keywords. Demonstrate how to check valid identifiers programmatically.

Q28. Explain the Python Data Model and Built-in Special (Dunder) Attributes: __name__, __doc__, __file__, __dict__, and __class__.

Q29. Design an interactive quadratic equation solver in Python that handles real roots, single degenerate roots, and complex conjugate roots with full error handling.

Q30. Explain the if __name__ == '__main__' idiom in Python. What problem does it solve, and how does Python execute scripts versus imported modules?

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[B]	**Q41:**	[B]	**Q61:**	[B]	**Q81:**	[A]
Q2:	[B]	**Q22:**	[B]	**Q42:**	[B]	**Q62:**	[A]	**Q82:**	[B]
Q3:	[D]	**Q23:**	[C]	**Q43:**	[B]	**Q63:**	[A]	**Q83:**	[B]
Q4:	[B]	**Q24:**	[C]	**Q44:**	[A]	**Q64:**	[C]	**Q84:**	[B]
Q5:	[B]	**Q25:**	[B]	**Q45:**	[B]	**Q65:**	[A]	**Q85:**	[B]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[B]	**Q66:**	[C]	**Q86:**	[B]
Q7:	[B]	**Q27:**	[D]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[B]
Q8:	[B]	**Q28:**	[B]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[B]
Q9:	[A]	**Q29:**	[B]	**Q49:**	[B]	**Q69:**	[B]	**Q89:**	[A]
Q10:	[B]	**Q30:**	[C]	**Q50:**	[B]	**Q70:**	[B]	**Q90:**	[A]
Q11:	[B]	**Q31:**	[D]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[B]
Q12:	[B]	**Q32:**	[B]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[B]
Q13:	[C]	**Q33:**	[B]	**Q53:**	[A]	**Q73:**	[B]	**Q93:**	[A]
Q14:	[C]	**Q34:**	[A]	**Q54:**	[B]	**Q74:**	[B]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[B]	**Q55:**	[B]	**Q75:**	[B]	**Q95:**	[A]
Q16:	[B]	**Q36:**	[B]	**Q56:**	[B]	**Q76:**	[B]	**Q96:**	[D]
Q17:	[B]	**Q37:**	[B]	**Q57:**	[A]	**Q77:**	[B]	**Q97:**	[B]
Q18:	[C]	**Q38:**	[A]	**Q58:**	[B]	**Q78:**	[A]	**Q98:**	[B]
Q19:	[C]	**Q39:**	[B]	**Q59:**	[C]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[B]	**Q40:**	[B]	**Q60:**	[B]	**Q80:**	[B]	**Q100:**	[B]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS programs. Compare CPython with compiled languages (C/C++) and explain the roles

5. Complexity Analysis:
- Time Complexity: O(1) arithmetic operations per instruction.
- Space Complexity: O(1) auxiliary memory for local variables and operand stack.

Solution to Question 2:

- Space Complexity: O(1) pointer reassignment overhead.

Solution to Question 3: Explain CPython's Small Integer Caching (-5 to 256) and String Interning mechanisms. Write code to verify memory identity (is) versus

>>>

5. Complexity Analysis:
- Value Equality (==): O(N) where N is number of elements/digits.
- Identity Comparison (is): O(1) single machine pointer register comparison.

Solution to Question 4: Explain Mutable vs Immutable data types in Python. Detail how memory allocation behaves during in-place mutations vs rebinding operations.

my_list.append(40) -> Size becomes 4. Address of list remains 0x2000. No new list object is allocated.

5. Complexity Analysis:
- List In-place Append: Amortized O(1) time complexity.
- String In-place Concat (when unshared): O(1) optimization in CPython; otherwise O(N) allocation and memory copy.

Solution to Question 5: >>> Explain Python Variable Naming Conventions (PEP 8) and NameError exceptions. Discuss common scenarios that trigger NameError a

```
safe_variable_lookup()
```

5. Complexity Analysis:
- Scope Lookup: O(1) average time via dictionary hash table lookups across Local, Global, and Builtins namespaces.
- Space Complexity: O(1) auxiliary overhead.

Solution to Question 6:

```
>>> # Solution to Question 6:
```

Solution to Question 57: Main Operator Precedence and Associativity in Python, Provide a full hierarchy table from highest to lowest precedence with illustrati

>>> Solution Statement Question Bank Explain the role of operators and operands in Python. How does Python evaluate expressions?

- Bitwise NOT (~): Inverts all bits.
- Left Shift (<<): Shifts bits left by n positions, equivalent to multiplying by 2^n: x << n = x * (2**n).
- Right Shift (>>): Shifts bits right by n positions, equivalent to floor dividing by 2^n: x >> n = x // (2**n).

2. Two's Complement Arithmetic & The ~x Identity:
In computer systems, negative integers are represented using two's complement notation:
- For any integer x, its two's complement negative representation -x is obtained by inverting all bits (~x) and adding 1:
-x = ~x + 1
- Rearranging this fundamental formula algebraically:
~x = -x - 1 = -(x + 1)
Therefore:
~0 = -(0 + 1) = -1
~5 = -(5 + 1) = -6
~(-8) = -(-8 + 1) = 7

3. Python Implementation & Bitwise Flag Demonstration:

```
# Permissions System Using Bitwise Flags
READ = 1 << 0 # 0001 (1)
WRITE = 1 << 1 # 0010 (2)
EXECUTE = 1 << 2 # 0100 (4)

user_perm = READ | WRITE # 0011 (3)

def check_permission(perm, flag):
return (perm & flag) != 0

print(f"User Permissions: bin={bin(user_perm)}")
print(f"Has Read Access: {check_permission(user_perm, READ)}")
print(f"Has Execute Access: {check_permission(user_perm, EXECUTE)}")

# Bitwise Shifts
val = 12
print(f"12 << 2 = {val << 2} (12 * 4)")
print(f"12 >> 2 = {val >> 2} (12 // 4)")
print(f"~12 = {~val} (Formula: -(12 + 1) = -13)")
```

4. Complexity Analysis:
- Time Complexity: O(1) for machine word bitwise operations.
- Space Complexity: O(1) integer registers.

>> Solution to Question 10: Explain String Indexing and Slicing in Python. Detail the syntax s[start:stop:step], positive vs negative indexing, boundary defaults, and

- start: The inclusive starting index. Defaults to 0 if step > 0, or len(s) - 1 if step < 0.
- stop: The exclusive ending boundary index. Defaults to len(s) if step > 0, or -1 (past beginning) if step < 0.
- step: The stride or increment between successive indices. Can be positive (forward traversal) or negative (backward traversal). Defaults to +1.

3. Special Slicing Patterns:
- s[:] -> Creates a shallow copy of the entire string.
- s[::2] -> Extracts every second character (even indices).
- s[1::2] -> Extracts characters at odd indices.
- s[::-1] -> Traverses the string in reverse direction, producing a reversed string.
- Boundary Handling: Slicing never raises an IndexError even if indices exceed string boundaries; Python automatically clamps indices to [0, len(s)].

4. Python Code Demonstration:

```
text = "ComputerScience"
print(f"Original String: '{text}' (Length: {len(text)})")

# Positive Indexing & Slicing
print(f"text[0:8] -> '{text[0:8]}'")
print(f"text[8:] -> '{text[8:]}'")
print(f"text[::2] -> '{text[::2]}'")

# Negative Indexing & Slicing
print(f"text[-7:] -> '{text[-7:]}'")
print(f"text[:-7] -> '{text[:-7]}'")
print(f"text[::-1] -> '{text[::-1]}' (Reversed)")

# Boundary Clamping Tolerance
print(f"text[5:100] -> '{text[5:100]}' (No IndexError raised)")
```

5. Complexity Analysis:
- Time Complexity: O(K) where K = len(slice) is the number of characters extracted to create the new string.
- Space Complexity: O(K) memory allocated for the new sliced string object.

>>P Solution Statement Question 11 Explain String Immutability and String Operations (+, *, in, not in), Why are strings designed to be immutable in Python, and how does t

- Security: Critical system resources (file paths, network sockets, SQL connection strings) cannot be maliciously altered after validation checks.

3. String Operations:
- Concatenation (+): Combines two strings into a new string: 'Py' + 'thon' -> 'Python'.
- Repetition (*): Replicates string n times: 'abc' * 3 -> 'abcabcabc'.
- Membership (in, not in): Substring search in O(N) worst-case time using two-way string search algorithm.

4. Python Code Demonstration:

```
base_str = "Python"
print(f"Initial String: {base_str}, ID: {id(base_str)}")

# Attempting mutation raises TypeError
try:
base_str[0] = 'J'
except TypeError as ex:
print(f"Caught Expected Mutation Error: {ex}")

# Concatenation creates a brand new string
new_str = base_str + " Programming"
print(f"New String: {new_str}, New ID: {id(new_str)}")

# Efficient Building via join() vs Inefficient Building via +=
words = ["Modern", "Operating", "Systems", "Design"]
efficient_sentence = " ".join(words)
print(f"Built Sentence: {efficient_sentence}")
```

5. Complexity Analysis:
- Inefficient += loop: O(N^2) time due to repeated reallocations and copies.
- str.join(list): O(N) linear time with single-pass memory pre-allocation.

Solution to Question 12: Discuss the built-in type conversion and type casting functions in Python (int, float, str, bool, complex). Explain explicit conversion versu

3. Python Code Demonstration:

```
# Implicit Coercion
int_val = 15
flt_val = 4.5
coerced_result = int_val + flt_val
print(f"Implicit Coercion: {int_val} ({type(int_val).__name__}) + {flt_val}
({type(flt_val).__name__}) = {coerced_result} ({type(coerced_result).__name__})")

# Explicit Conversions
binary_str = "1101"
decimal_from_bin = int(binary_str, 2)
hex_from_str = int("1A", 16)
print(f"Binary '1101' to Int: {decimal_from_bin} | Hex '1A' to Int: {hex_from_str}")

trunc_float = int(9.99)
print(f"int(9.99) Truncation: {trunc_float}")

# Float to String and String to Complex
cmp_obj = complex("3+4j")
print(f"Parsed Complex Object: {cmp_obj}, Conjugate: {cmp_obj.conjugate()}")
```

4. Complexity Analysis:

- Time Complexity: O(1) for primitive numeric conversions; O(D) where D is number of string digits during base conversions.
- Space Complexity: O(1) auxiliary space.

>> Solution to Question 13: Explain Python Comments, Docstrings, and the Composition of Statements. How are docstrings accessed at runtime via `__doc__` and

```
    height (float): The axial length/height of the cylinder.

    Returns:
    float: Computed volume in cubic units (V = pi * r^2 * h).
    """
    import math
    # Compute base area then multiply by axial height
    base_area = math.pi * (radius ** 2)
    volume = base_area * height
    return volume

    # Runtime Docstring Introspection
    print("Introspecting __doc__ attribute directly:")
    print(calculate_cylinder_volume.__doc__)

    print(f"Calculated Volume (r=3, h=5): {calculate_cylinder_volume(3.0, 5.0):.4f}")
```

5. Complexity Analysis:
- Docstring Overhead: Stored once as an immutable string object in module const table; zero runtime CPU penalty during function calls.

Solution to Question 14:

>>> Explain Python 2 versus Python 3 architectural differences. Focus on print, Unicode string handling, integer division, and range vs xran

3. Python 3 Code Verification:

```
import sys

# Division behavior in Python 3
print(f"5 / 2 in Python 3 = {5 / 2} (type: {type(5 / 2).__name__})")
print(f"5 // 2 in Python 3 = {5 // 2} (type: {type(5 // 2).__name__})")

# Range Memory Efficiency
r = range(10_000_000)
print(f"Size of range(10,000,000) object: {sys.getsizeof(r)} bytes (O(1) memory!)")

# Unicode Text vs Bytes
unicode_text = "Python Programming ?" # Greek Omega
byte_data = unicode_text.encode('utf-8')
print(f"Unicode str: {unicode_text} | Encoded bytes: {byte_data}")
```

4. Complexity Analysis:
- Memory Complexity of range(N) in Python 3: O(1) fixed memory.
- Division Time Complexity: O(1).

>> Solution to Question 15: Explain Python Virtual Environments (venv) and Package Management (pip). Why are virtual environments essential in enterprise softw

```
pip install -r requirements.txt

# Step 6: Deactivate the environment
deactivate
```

4. Production Best Practices:
- Always commit requirements.txt or pyproject.toml to Git; NEVER commit the venv folder itself.
- Leverage modern tools like poetry or pipenv for deterministic lockfile resolution.

5. Complexity Analysis:
- Runtime Overhead: Zero execution overhead; environment isolation is purely a PATH redirection mechanism.

Solution to Question 16

>>> Explain the Python Standard Library and the 'Batteries Included' philosophy. Provide examples of 5 essential built-in modules (math, sy

```
dice_roll = random.randint(1, 6)
print(f"Random Dice Roll: {dice_roll}")

# 5. time demonstration
start_t = time.perf_counter()
_ = [x ** 2 for x in range(100_000)]
elapsed_ms = (time.perf_counter() - start_t) * 1000
print(f"Benchmark: 100k squares calculated in {elapsed_ms:.2f} ms")
```

4. Complexity Analysis:
- Time Complexity: O(1) for math, os, and random generator calls; O(N) for list comprehension.
- Space Complexity: O(N) for 100,000 integer elements.

>>Solution to Question 17: Explain Modern Python String Formatting Paradigms: % formatting, str.format(), and Formatted String Literals (F-Strings). Compare per

```
# Self-documenting F-string (Python 3.8+)
x = 25
print(f"Self-documenting: {x*2=}")
```

4. Complexity Analysis:
- Time Complexity: O(N) where N is output string length.
- Space Complexity: O(N) memory allocated for formatted string.

Solution to Question 18:

>> Explain CPython's Reference Counting and Cyclic Garbage Collection mechanisms. How are reference cycles detected and resolved?

```
unreachable_count = gc.collect()
print(f"Cyclic GC collected {unreachable_count} unreachable cyclic objects.")
```

4. Complexity Analysis:
- Reference Counting: O(1) amortized increment/decrement per assignment.
- Cyclic GC Collection: O(N) where N is number of candidate container objects in the generation.

Solution to Question 19:

>>> Explain Python's sys.argv and Command-Line Argument Processing. Write a standalone Python script demonstrating command-line pa

- Space Complexity: O(A) where A is the total string length of passed arguments.

Solution to Question 20:

>>> Explain Floating-Point Representation (IEEE 754) in Python. Why does 0.1 + 0.2 != 0.3, and how should financial/scientific calculations

1. Problem Context:
Engineering applications frequently require seamless conversion across thermodynamic temperature scales: Celsius (C), Fahrenheit (F), and Kelvin (K). The program must validate inputs (e.g., rejecting temperatures below Absolute Zero: -273.15 C or 0 K).

2. Thermodynamic Formulas:
- F = (C * 9/5) + 32
- C = (F - 32) * 5/9
- K = C + 273.15
- C = K - 273.15

3. Python Implementation:
class TemperatureConverter:
ABSOLUTE_ZERO_C = -273.15
ABSOLUTE_ZERO_K = 0.0

@staticmethod
def celsius_to_all(celsius: float):
if celsius < TemperatureConverter.ABSOLUTE_ZERO_C:
raise ValueError(f"Physics Violation: Temperature {celsius} C is below Absolute Zero (-273.15 C)!")

fahrenheit = (celsius * 9 / 5) + 32
kelvin = celsius + 273.15
return {"Celsius": celsius, "Fahrenheit": round(fahrenheit, 2), "Kelvin": round(kelvin, 2)}

@staticmethod
def fahrenheit_to_all(fahrenheit: float):
celsius = (fahrenheit - 32) * 5 / 9
return TemperatureConverter.celsius_to_all(celsius)

Demonstration and Verification
try:
c_sample = 37.0 # Human body temperature
res = TemperatureConverter.celsius_to_all(c_sample)
print(f"Human Body Temp: {res['Celsius']} C = {res['Fahrenheit']} F = {res['Kelvin']} K")

Boundary test: Absolute zero
abs_zero = TemperatureConverter.celsius_to_all(-273.15)
print(f"Absolute Zero: {abs_zero['Celsius']} C = {abs_zero['Fahrenheit']} F = {abs_zero['Kelvin']} K")

Invalid temperature test
TemperatureConverter.celsius_to_all(-300.0)
except ValueError as err:
print(f"Validation Caught: {err}")

4. Memory Model & Complexity:
- Time Complexity: O(1) elementary arithmetic operations.
- Space Complexity: O(1) dictionary creation.

>> Solution to Question 23:

[Problem Statement]: Implement a comprehensive Python script demonstrating all Bitwise Operations to manipulate hardware register control flag
[Detailed Step-by-Step Answer]:
1. Problem Context:
In embedded systems and low-level driver development, hardware status registers are manipulated
using bitwise bit masks:
- Set Bit: register |= (1 << n)
- Clear Bit: register &= ~(1 << n)
- Toggle Bit: register ^= (1 << n)
- Check Bit: (register & (1 << n)) != 0

2. Python Implementation:

```
class HardwareRegister:
def __init__(self, initial_val=0x00):
self._reg = initial_val & 0xFF # Emulate 8-bit hardware register

def set_bit(self, bit_pos: int):
self._reg |= (1 << bit_pos)

def clear_bit(self, bit_pos: int):
self._reg &= ~(1 << bit_pos)

def toggle_bit(self, bit_pos: int):
self._reg ^= (1 << bit_pos)

def is_bit_set(self, bit_pos: int) -> bool:
return (self._reg & (1 << bit_pos)) != 0

def display(self, label=""):
print(f"{label:<25} | Hex: 0x{self._reg:02X} | Bin: {self._reg:08b}")

# Demonstration
reg = HardwareRegister(0b00000000)
reg.display("Initial State")

# Set Bit 3 (e.g., Enable Motor)
reg.set_bit(3)
reg.display("After Setting Bit 3")

# Set Bit 0 (e.g., Power ON)
reg.set_bit(0)
reg.display("After Setting Bit 0")

# Check Bit 3
print(f"Is Bit 3 Active? {reg.is_bit_set(3)}")

# Toggle Bit 0 (Turn Power OFF)
reg.toggle_bit(0)
reg.display("After Toggling Bit 0")

# Clear Bit 3 (Disable Motor)
reg.clear_bit(3)
reg.display("After Clearing Bit 3")
```

3. Complexity Analysis:

- Time Complexity: O(1) single-cycle bitwise ALU operations.
- Space Complexity: O(1) 8-bit integer state.

Solution to Question 24:

```
>>> # Solution to Question 24: Implement a Python program to perform Cartesian Coordinate System operations: Distance calculation, midpoint, polar coordinate conv
```

```
print(f"Slope of Line P1-P2: {p1.slope_to(p2):.4f}")
```

3. Complexity Analysis:
- Time Complexity: O(1) analytic geometry operations.
- Space Complexity: O(1) object allocations.

Question 25: Explain the Walrus Operator (:=) introduced in Python 3.8. Discuss its motivation, syntax, performance advantages, and write code demonstrating its use in a while loop and a list comprehension.

Solution:

- Time Complexity: Eliminates redundant O(N) or O(Cost) duplicate evaluations.
- Space Complexity: O(1) auxiliary variable binding.

Python keywords are reserved tokens defined in the interpreter's grammar. They have special syntactic meaning (e.g., if, for, class, def, return, yield) and cannot be assigned as variable, function, or class names.
Modern Python 3.10+ contains 35 keywords.

2. Dynamic Inspection via keyword Module:
The standard library keyword module provides:
- keyword.kwlist: A sorted list of all reserved keywords.
- keyword.iskeyword(s): Returns True if string s is a reserved keyword.
- keyword.issoftkeyword(s): Identifies contextual soft keywords introduced in Python 3.10 (e.g., match, case).
- str.isidentifier(): Built-in string method checking if string follows valid identifier syntax.

3. Python Implementation & Identifier Validator:

```
import keyword

def validate_python_identifier(identifier_name: str) -> tuple:
"""Validates whether a string can serve as a legal variable name."""
if not isinstance(identifier_name, str):
return (False, "Identifier must be a string.")

if len(identifier_name) == 0:
return (False, "Identifier cannot be empty.")

if keyword.iskeyword(identifier_name):
return (False, f"'{identifier_name}' is a reserved Python keyword.")

if not identifier_name.isidentifier():
return (False, f"'{identifier_name}' contains illegal characters or starts with a digit.")

return (True, f"'{identifier_name}' is a completely valid Python identifier.")

# Test Scenarios
test_cases = ["student_name", "_temp_var", "2nd_rank", "class", "total$sum", "def",
"normalVariable"]
print(f"Total Python Keywords: {len(keyword.kwlist)}")
print("-" * 60)
for tc in test_cases:
valid, msg = validate_python_identifier(tc)
status = "[VALID] " if valid else "[INVALID]"
print(f"{status} {msg}")
```

4. Complexity Analysis:
- Time Complexity: O(1) hash set lookup in keyword.iskeyword().
- Space Complexity: O(1) auxiliary validation overhead.

Solution to Question 28:
>>> Explain the Python Data Model and Built-in Special (Dunder) Attributes: __name__, __doc__, __file__, __dict__, and __class__.

('dunder') attributes attached to modules, classes, and instances.

2. Core Dunder Attributes:
1. __name__: Holds the name of the module or function. For the main entry script executed directly, __name__ is set to '__main__'.
2. __doc__: Stores the documentation string (docstring) of the entity.
3. __file__: Contains the absolute filesystem path from which the module was loaded.
4. __dict__: A dictionary namespace holding all writable attributes of an object or module.
5. __class__: A reference to the type or class of which the current object is an instance.

3. Python Demonstration Code:
```
"""Sample Module Docstring for Introspection Demonstration."""

class StudentProfile:
"""Student demographic and academic record class."""
def __init__(self, name: str, gpa: float):
self.name = name
self.gpa = gpa

stu = StudentProfile("Meera", 3.88)

print("Introspecting Module & Object Dunder Attributes:")
print(f"Module __name__: {__name__}")
print(f"Class __name__: {StudentProfile.__name__}")
print(f"Class __doc__: {StudentProfile.__doc__}")
print(f"Instance __class__: {stu.__class__}")
print(f"Instance __dict__: {stu.__dict__}")

# Modifying state through __dict__
stu.__dict__['major'] = 'Computer Science'
print(f"Dynamic Attribute Added via __dict__: stu.major = {stu.major}")
```

4. Complexity Analysis:
- Attribute Access: O(1) dictionary lookup in instance and class __dict__.
- Space Complexity: O(K) where K is number of instance attributes stored in __dict__.

>> Solution to Question 29: Here is an interactive quadratic equation solver in Python that handles real roots, single degenerate roots, and complex conjugate roots

```
if a == 0:
raise ValueError("Degenerate Case: 'a' cannot be zero in a quadratic equation.")

discriminant = (b ** 2) - (4 * a * c)
print(f"\nSolving: ({a})x^2 + ({b})x + ({c}) = 0 | Discriminant = {discriminant}")

if discriminant > 0:
sqrt_d = math.sqrt(discriminant)
r1 = (-b + sqrt_d) / (2 * a)
r2 = (-b - sqrt_d) / (2 * a)
print(f" Two Distinct Real Roots: x1 = {r1:.4f}, x2 = {r2:.4f}")
return (r1, r2)
elif discriminant == 0:
r = -b / (2 * a)
print(f" Single Degenerate Real Root: x = {r:.4f}")
return (r,)
else:
# Complex conjugate roots using cmath
sqrt_d = cmath.sqrt(discriminant)
r1 = (-b + sqrt_d) / (2 * a)
r2 = (-b - sqrt_d) / (2 * a)
print(f" Complex Conjugate Roots: x1 = {r1.real:.4f} + {r1.imag:.4f}j, x2 = {r2.real:.4f} +
{r2.imag:.4f}j")
return (r1, r2)

# Test All Three Regimes
solve_quadratic_equation(1, -5, 6) # Roots: 3.0, 2.0 (Delta = 1)
solve_quadratic_equation(1, -4, 4) # Roots: 2.0 (Delta = 0)
solve_quadratic_equation(1, 2, 5) # Complex Roots: -1 +/- 2j (Delta = -16)

3. Complexity Analysis:
- Time Complexity: O(1) elementary arithmetic and square root computation.
- Space Complexity: O(1) memory for roots.
```

>>Solution to Question 30

Solution to Question 30

Explain the if __name__ == '__main__' idiom in Python. What problem does it solve, and how does Python execute scripts versus impo

3. The Guard Idiom:
By enclosing script entry points within the guard:
if __name__ == '__main__':
main()
developers guarantee that execution occurs ONLY when the script is run directly, while allowing all classes, functions, and constants to be cleanly imported elsewhere without side effects.

4. Python Code Demonstration:

```
def calculate_factorial(n: int) -> int:
"""Reusable library function."""
if n < 0:
raise ValueError("Factorial undefined for negative numbers.")
result = 1
for i in range(1, n + 1):
result *= i
return result

def main():
"""CLI entry point for direct script execution."""
print("Executing as standalone script...")
test_val = 5
print(f"Test Factorial({test_val}) = {calculate_factorial(test_val)}")

# Idiomatic Module Guard
if __name__ == '__main__':
main()
else:
print(f"Module '{__name__}' imported successfully into another namespace.")
```

5. Complexity Analysis:
- Time Complexity: O(1) string equality comparison during module initialization.
- Space Complexity: O(1) interpreter state.
